A common gripe that students express is how much they feel the work they do in class fails to map to the real world. Now that you've had a chance to complete the portfolio project, how much better (or worse) do you think you understand software development and why?

I feel like I understand more of how OSU envisions software development, but I still have lots of confusion regarding the "OSU Discipline" as, despite my best efforts, I cannot seem to get any points on it. It was good practice to design a complex program from scratch, however.

Also, did the portfolio project surface any gaps in your own knowledge of software development. If so, what are those gaps and how did you address them?

This project demonstrated that I have some really bad slip-ups with mathematics quite often. Trying to do arithmetic on Exponent-Mantissa notated numbers isn't easy, but I made lots of small errors that completely ruined several functions and went under the radar of my JUnit tests for several iterations. On that note, I can't seem to make great unit tests either!

Finally, as a part of completing the portfolio project, to what extent has your perspective of software development changed, if at all? In other words, is software development something you still enjoy? If not, why not?

Despite my gripes with the project, nothing will dissuade me from pursuing software development. Seeing unit tests light up green, seeing the actual results you want show up when you start utilizing what you've made, they're all high points I rarely get with other hobbies of mine. I do realize now that I have a hard time following rules, especially if I don't "agree" with them, and struggle to conform my code to someone else's discipline.

One of the challenges of completing the portfolio project is picking up a lot of skills on your own. Some of these skills are, of course, software skills. However, there are plenty of other skills you may have picked up through this process. Therefore, the first question is what skills did you pick up through this process?

I learned how to program in crunch time, how to give feedback on others' code I've never seen beforehand on entirely separate topics, and how to use version control (GitHub) and why it's useful to have.

The follow-up question is: could you rephrase these skills you picked up as bullet points that you could put on a resume? Try it below.

- Proficient in version control systems (GitHub)

- Rapid debugging and refactoring of projects
- Can provide effective, detailed feedback on foreign code written by others

Next, how has working on this project affected your career trajectory? In other words, do you now hate the topic you picked? Or, are you even more interested in it? Both outcomes are valuable to your personal development.

If anything, I'm disappointed in myself for not realizing my project to the fullest. I do tend to have overly ambitious ideas, but this one was so tantalizing that I am ashamed to have it in an incomplete state as it is now. If anything, it makes me wish for an even longer-form project, something that takes months of continuous, complex work, and hopefully working with a discipline less vague in the process.

Finally, consider the skills you've picked up and your current career trajectory. What are some things you could do to continue on your career trajectory? Also, who are some mentors you could contact to help you stay on your path?

I need to work (and finish) personal projects like these. I believe I have the ability to creatively design software and eventually create something tangible, but I need more examples before I continue my career. More importantly, I need to get a better grasp on relevant computer science mathematics, so my designs are not only creative, but efficient on hardware. I would hope that perhaps Professor Jeremy Griftski and Professor K.T. Vandergriff may offer mentorship to strengthen my real-world design philosophy and computer science mathematic fundamentals respectively.